

Módulo 11 – Capítulo 03 – Sección 01

El desafío del legado: datos en silos, APIs inconsistentes y formatos heterogéneos

Hay una brecha conceptual entre la arquitectura de referencia descrita en el capítulo anterior y la realidad tecnológica de la mayoría de los enterprises: mientras la arquitectura de referencia asume servicios con APIs REST bien documentadas, datos en formatos estructurados accesibles en tiempo real, y sistemas con SLAs de disponibilidad definidos, los sistemas que realmente contienen los datos de negocio más valiosos suelen ser exactamente lo opuesto. Son sistemas legacy — sistemas con más de diez años de antigüedad, con frecuencia sin documentación actualizada, desarrollados en tecnologías fuera de soporte — que contienen décadas de transacciones, contratos, historiales de clientes, y registros operacionales que ningún sistema de IA enterprise puede ignorar.

El valor de los datos legacy para los sistemas de IA es inversamente proporcional a la facilidad de acceso a ellos. Los datos más ricos en información de negocio son frecuentemente los más difíciles de extraer: el historial de contratos de los últimos 20 años está en un sistema de gestión documental con una API SOAP de 2003 que solo funciona desde la intranet; el historial de transacciones de clientes está en una base de datos Oracle gestionada por el equipo de mainframe, accesible solo mediante procedimientos almacenados con esquemas no documentados; y las reglas de negocio más complejas están codificadas en programas COBOL que nadie en la organización puede modificar sin arriesgarse a romper el proceso de facturación.

Los silos de datos emergen históricamente porque cada sistema enterprise fue construido para optimizar su función específica sin considerar la interoperabilidad. El ERP (SAP, Oracle E-Business Suite) mantiene los datos maestros de clientes con un identificador propio que no coincide con el identificador del CRM (Salesforce), que a su vez difiere del identificador del sistema de facturación legacy. Reconciliar estas tres versiones del mismo cliente, cada una con su modelo de datos propio y sus reglas de actualización independientes, puede convertirse en un proyecto de data engineering de meses antes de que el sistema de IA pueda recibir un contexto coherente sobre ese cliente.

La heterogeneidad de formatos añade una capa adicional de complejidad. En un solo enterprise, el AI Engineer puede encontrarse integrando datos en JSON desde la API REST del CRM moderno, XML desde el sistema SOAP del ERP, fixed-width files desde el mainframe COBOL (donde la posición de cada campo en el archivo tiene significado semántico que no está documentado en ningún lugar visible), EDI X12 desde las comunicaciones con proveedores, HL7 FHIR desde el sistema de historial clínico en el sector salud, y PDF no estructurado desde la base de documentos de contratos escaneados. Normalizar todos estos formatos en una representación que un LLM pueda consumir coherentemente es, en sí mismo, un proyecto de ingeniería sustancial.

Nota del Arquitecto: *El primer paso en cualquier integración con legacy no es técnico sino de descubrimiento. El mapa de dependencias de datos — qué sistemas existen, qué datos contienen, cómo se relacionan entre sí, quién los mantiene, y cuáles son sus restricciones de acceso y disponibilidad — es el artefacto que debe existir antes de cualquier decisión de arquitectura. Sin ese mapa, la integración introduce riesgos que no pueden cuantificarse, porque no se conoce el sistema que se está integrando.*

Puntos críticos de los sistemas legacy

- **Ausencia de APIs REST:** la única interfaz disponible puede ser un endpoint SOAP con WSDL de 2003, un procedimiento almacenado en Oracle que retorna un cursor de base de datos, o un job batch que genera un archivo CSV cada 24 horas en un servidor de ficheros con acceso SFTP.
- **Esquemas no documentados:** tablas con nombres como T_MSTDAT_001 y columnas como COD_FLAG_X que requieren ingeniería inversa mediante análisis de código fuente COBOL o conversaciones con el equipo de mantenimiento del sistema que concentra ese conocimiento.
- **Calidad de datos degradada:** duplicados sin reconciliar, valores nulos en campos nominalmente obligatorios, fechas en formatos inconsistentes dentro del mismo campo (YYYYMMDD en algunos registros, DD/MM/YYYY en otros), y encoding de caracteres mezclado que produce caracteres corruptos al hacer ETL.
- **Dependencias ocultas:** cambiar el formato de un campo en el sistema legacy puede romper silenciosamente 15 sistemas downstream que nadie recuerda que dependen de ese campo exacto en esa posición del fixed-width file — porque esas dependencias nunca se documentaron.

- **Restricciones de disponibilidad:** ventanas de mantenimiento nocturnas donde el sistema no está disponible para extracción de datos, y prohibiciones de carga adicional durante el horario de negocio por riesgo de impactar el proceso productivo principal.
-

Para recordar: El primer paso para integrar IA con sistemas legacy es producir un mapa de dependencias de datos actualizado — no el mapa que el equipo de TI tiene documentado desde 2015, sino el mapa real del estado actual, verificado mediante análisis técnico. Sin ese mapa, cualquier integración introduce riesgos que no pueden cuantificarse.

La sección siguiente ofrece los patrones de diseño específicos que permiten encapsular la complejidad de los sistemas legacy detrás de interfaces modernas: el Adapter, la Facade, y el Anti-Corruption Layer.

Módulo 11 – Capítulo 03 – Sección 02

Patrones de integración: Adapter, Facade y Anti-Corruption Layer para sistemas legacy

La integración con sistemas legacy sin patrones de diseño explícitos produce inevitablemente uno de dos problemas: el acoplamiento directo, donde el sistema de IA depende de los detalles de implementación del legacy (sus formatos de datos, sus convenciones de nomenclatura, sus reglas de negocio implícitas) y debe modificarse cada vez que el legacy cambia; o la duplicación, donde la lógica de transformación de datos legacy se replica en múltiples partes del sistema de IA sin un lugar centralizado donde mantenerla. Los patrones de integración del Domain-Driven Design y de los Enterprise Integration Patterns (EIP) ofrecen soluciones verificadas a ambos problemas, y su aplicación al contexto específico de sistemas de IA enterprise permite construir integraciones que son robustas ante los cambios en los sistemas subyacentes.

El **patrón Adapter** es la solución más directa: traduce la interfaz de un sistema legacy a una interfaz que el sistema de IA puede consumir sin acoplarse a los detalles del legacy. Un Adapter SOAP-to-REST, implementado con Apache Camel, MuleSoft, o un AWS API Gateway con mapping templates, convierte las llamadas SOAP/XML de un sistema SAP en endpoints RESTful con payloads JSON, absorbiendo la conversión de formato y el manejo de errores específicos del protocolo SOAP — los faults, los timeouts de la conexión SOAP, y los esquemas XML con namespaces anidados — sin que el sistema de IA tenga que saber que el sistema subyacente habla SOAP. Cuando SAP actualiza su API SOAP, solo el Adapter necesita actualizarse; el sistema de IA y todos los demás consumidores continúan hablando REST sin interrupción.

El **patrón Facade** es más ambicioso: agrega múltiples sistemas legacy detrás de una única interfaz unificada. En un sistema de RAG que necesita contexto sobre un cliente para responder preguntas, la Facade puede orquestar internamente consultas al ERP (para datos de facturación), al CRM (para datos de contacto y historial de interacciones), y al sistema de gestión documental (para contratos activos), devolviendo al sistema de IA un objeto de contexto unificado del cliente sin que el sistema de IA tenga que saber qué sistemas fueron

consultados. La ventaja no es solo de simplicidad: cuando se agrega un cuarto sistema fuente (por ejemplo, el sistema de soporte técnico para el historial de tickets), la Facade absorbe la integración nueva sin que el sistema de IA necesite modificarse.

El **Anti-Corruption Layer (ACL)** es el patrón más crítico para sistemas de IA enterprise. Un legacy que lleva décadas en producción acumula inconsistencias en su modelo de datos que son normales dentro del sistema — una dirección de cliente puede estar en tres formatos distintos dependiendo de cuándo fue creado el registro — pero que, si se propagan al sistema de IA sin normalización, contaminan el conocimiento del LLM con inconsistencias que producen respuestas incoherentes. El ACL actúa como una capa de traducción bidireccional que normaliza los datos del legacy antes de que entren al pipeline de IA, aplicando las reglas de dominio que el sistema de IA espera. En Python, esto se implementa con Pydantic validators que rechazan activamente los datos que no cumplen el modelo de dominio del sistema de IA, forzando la detección de discrepancias en el momento de la integración en lugar de en el momento de la generación de la respuesta.

Aspectos técnicos de los patrones de integración

- **Adapter SOAP-to-REST:** implementado con Apache Camel (Java), MuleSoft Anypoint, o AWS API Gateway con mapping templates que convierten WSDLs en OpenAPI specs; herramientas como soap2rest o Postman pueden generar el mapping inicial de manera semi-automática.
- **Facade de agregación:** microservicio que implementa el patrón Backend-for-Frontend (BFF) específico para IA, consolidando datos de múltiples fuentes legacy en un esquema de contexto optimizado para el formato de entrada del LLM, reduciendo el tamaño del contexto al eliminar campos innecesarios para el caso de uso.
- **Anti-Corruption Layer con Pydantic:** modelos de datos del dominio de IA con validaciones estrictas (tipos, rangos, formatos) que rechazan activamente los datos del legacy que no cumplen las invariantes del dominio, con logging de todos los rechazos para detectar tendencias de calidad de datos.
- **Change Data Capture (CDC) con Debezium:** en lugar de polling periódico a las tablas del legacy, Debezium monitorea el transaction log de MySQL, PostgreSQL, u Oracle, publicando cada cambio en Kafka en tiempo real y sin impacto de rendimiento en la base de datos fuente.
- **Circuit breaker para legacy:** Resilience4j o Polly con timeouts agresivos (2-3 segundos máximo para sistemas legacy con SLAs débiles) y fallbacks a datos cacheados o respuestas degradadas cuando el sistema legacy no responde dentro del tiempo esperado.

Principio rector: Aislar la complejidad del legado en capas de integración explícitas permite que el sistema de IA evolucione independientemente de los cambios — o la falta de cambios — en los sistemas legacy subyacentes. La disciplina de no permitir que las abstracciones del legacy crucen la frontera del ACL es lo que hace que este principio sea sostenible a largo plazo.

Los patrones de integración resuelven el problema de cómo extraer datos del legacy. La sección siguiente aborda el problema de cómo transformarlos: la decisión entre ETL y ELT, y sus implicaciones específicas para los pipelines que alimentan sistemas de RAG.

Módulo 11 – Capítulo 03 – Sección 03

ETL vs ELT para datos empresariales que alimentan sistemas de IA

La decisión entre ETL y ELT para los pipelines de datos que alimentan sistemas de IA enterprise tiene consecuencias que van más allá de la preferencia tecnológica: determina la latencia con la que los nuevos datos están disponibles para el sistema de IA, la capacidad del equipo para iterar sobre la transformación de datos sin tocar la ingesta, y los costos operacionales a escala cuando el volumen de datos crece. Es una decisión de arquitectura que afecta la velocidad del ciclo de mejora del sistema de IA y que, una vez tomada, es costosa de cambiar.

En el modelo ETL clásico, la transformación ocurre antes de cargar los datos en el destino. Herramientas como Informatica PowerCenter, IBM DataStage, o Talend ejecutan las transformaciones en servidores intermedios especializados, garantizando que solo datos limpios y conformados llegan al data warehouse. Esta garantía tiene un precio: los datos crudos del sistema legacy no están accesibles para análisis exploratorio, las transformaciones deben ser definidas de manera exhaustiva antes de que los datos lleguen al destino, y cualquier cambio en la lógica de transformación requiere un ciclo de desarrollo y despliegue que introduce latencia en la disponibilidad de los datos actualizados.

El modelo ELT, adoptado por los data warehouses cloud modernos (Snowflake, BigQuery, Redshift), invierte el orden: los datos crudos se cargan primero en el data lake en su formato original, y las transformaciones se ejecutan después, directamente en el sistema de destino mediante SQL. La herramienta que ha popularizado este enfoque es dbt (data build tool), que trata las transformaciones SQL como código: las versiona en Git, las documenta con tests de calidad (not_null, unique, accepted_values), y las ejecuta de manera reproducible sobre los datos crudos. Para sistemas de IA, la ventaja clave del ELT es la disponibilidad temprana de los datos crudos: el AI Engineer puede explorar los datos originales para identificar features no anticipadas en el diseño inicial del pipeline, sin esperar a que el equipo de datos actualice las transformaciones.

Para sistemas de RAG enterprise, el pipeline de datos tiene una etapa adicional que ninguno de los dos modelos contempla nativamente: el chunking y embedding de los documentos procesados, seguido de la indexación en la base de datos vectorial. Esta etapa debe ejecutarse de manera incremental — solo reindexar los documentos que han cambiado desde la última indexación — porque reconstruir el índice vectorial completo para cada actualización es prohibitivamente costoso en tiempo y en llamadas a la API de embeddings. El mecanismo de detección de cambios más eficiente es el hash del contenido del documento: si el hash SHA-256 del documento actual coincide con el hash almacenado en la indexación anterior, el documento no necesita reindexarse.

La integración con Change Data Capture (CDC) mediante Debezium — mencionado en la sección anterior — complementa el ELT para datos de bases de datos legacy: en lugar de ejecutar el pipeline completo periódicamente, Debezium publica cada cambio en Kafka en tiempo real, y los workers de procesamiento consumen esos cambios y actualizan tanto el data warehouse como el índice vectorial de manera incremental, con una latencia de disponibilidad de segundos en lugar de horas.

Componentes críticos de la decisión ETL vs ELT

- **Latencia de datos:** ETL introduce latencia adicional por el paso de transformación intermedio; ELT permite que los datos crudos estén disponibles en minutos mientras las transformaciones se ejecutan en background sobre el data warehouse.
- **Capacidad de exploración:** ELT permite a los AI Engineers explorar los datos crudos en el data lake para identificar features nuevas sin modificar el pipeline de transformación existente, acelerando el ciclo de experimentación.
- **dbt para transformaciones versionadas:** transformaciones SQL en Git con tests de calidad integrados, documentación automática del linaje de datos, y ejecución reproducible que garantiza que los datos del data warehouse son siempre el resultado de transformaciones verificables.
- **Orquestación de pipelines:** Apache Airflow para pipelines batch complejos con DAGs, Prefect para flujos más simples con Python nativo con gestión de estado y reintentos automáticos, o Dagster para pipelines con énfasis en observabilidad de datos y linaje.
- **Calidad de datos antes de IA:** Great Expectations o Soda Core para validar contratos de datos en cada etapa del pipeline, fallando el job antes de que datos corruptos contaminen los índices vectoriales — previniendo que el problema de calidad llegue al usuario a través de una respuesta incorrecta del sistema de IA.

Buena práctica: Para sistemas de RAG enterprise, el pipeline ELT debe incluir detección de cambios en documentos fuente mediante hashing del contenido (MD5 o SHA-256) para reindexar solo los documentos modificados, no reconstruir el índice vectorial completo en cada actualización. A escala de 100.000 documentos, la diferencia entre reindexación incremental y completa es la diferencia entre un pipeline que tarda 5 minutos y uno que tarda 8 horas.

Con los patrones de extracción y transformación de datos legacy claros, la sección siguiente aborda los protocolos de comunicación en detalle: REST, SOAP, gRPC, y message queues, con criterios claros de cuándo usar cada uno.

Módulo 11 – Capítulo 03 – Sección 04

APIs de integración: REST, SOAP, gRPC y message queues para conectar el legado

El protocolo de comunicación que usa un sistema legacy no es una preferencia — es un hecho histórico. Un sistema bancario construido en 2002 habla SOAP porque SOAP era el estándar enterprise en 2002. Un sistema de gestión documental construido en 2015 expone REST porque REST era el estándar moderno en 2015. Un microservicio construido en 2023 puede usar gRPC porque gRPC ofrece ventajas de rendimiento que justifican la complejidad adicional para comunicaciones de alta frecuencia intra-cluster. El AI Engineer enterprise debe ser capaz de integrar con todos estos protocolos sin insistir en que los sistemas legacy adopten el protocolo que el equipo de IA preferiría, porque esa preferencia no cambia la realidad: el sistema legacy habla lo que habla, y el tiempo de migración no está bajo el control del equipo de IA.

REST/JSON es el protocolo de integración por defecto para sistemas modernos y el de menor fricción: las herramientas de testing (Postman, HTTPie), los clientes HTTP en cualquier lenguaje, y el soporte de API management enterprise (Kong, APIM) son maduros y bien documentados. Para sistemas de RAG, la compresión gzip del payload HTTP es especialmente relevante: los fragmentos de contexto que el sistema de IA necesita enviar a la API del legacy o recibir de ella pueden contener textos largos que se comprimen bien con gzip, reduciendo el consumo de ancho de banda y la latencia de serialización.

SOAP/XML requiere un esfuerzo adicional de integración: el cliente SOAP en Python más usado es zeep, que puede generar un cliente Python a partir del WSDL de manera automática, abstrayendo la complejidad de los schemas XML y los envelopes SOAP. Para sistemas de mayor escala, Apache Camel proporciona componentes de integración SOAP maduros y con soporte para los patrones de EIP (retry, error handling, dead letter channel). En ningún caso el sistema de IA debe consumir directamente una API SOAP si puede evitarse: la capa de Adapter o de Facade debe absorber la complejidad del protocolo SOAP y exponer una interfaz REST al sistema de IA.

gRPC con Protocol Buffers es el protocolo de elección para la comunicación intra-cluster de alta frecuencia entre los componentes del sistema de IA: entre el servicio de orquestación y el servicio de embeddings, o entre el servicio de orquestación y el LLM serving con vLLM. Las ventajas son concretas: la serialización binaria de Protocol Buffers es 5-10 veces más eficiente que JSON para el mismo payload, el soporte de streaming bidireccional permite enviar tokens de respuesta del LLM a medida que se generan (en lugar de esperar la respuesta completa), y los .proto schemas versionados en un schema registry garantizan compatibilidad backward entre versiones de clientes y servidores.

Message queues — RabbitMQ, Apache Kafka, AWS SQS — son la integración adecuada cuando el sistema legacy no puede ser consultado de manera sincrónica o cuando el volumen de eventos supera la capacidad de procesamiento en tiempo real. El sistema de IA actúa como consumidor de mensajes que el legacy publica cuando ocurren eventos relevantes: nueva factura creada, contrato firmado, ticket de soporte abierto. Esta arquitectura tiene la ventaja de no imponer carga adicional al sistema legacy más allá de la publicación del evento, y de permitir que el sistema de IA procese los eventos a su propio ritmo sin afectar la disponibilidad del sistema fuente.

Aspectos técnicos de cada protocolo

- **REST/JSON:** protocolo de integración por defecto con OpenAPI 3.0 para documentación, HTTP/2 para multiplexing de múltiples peticiones en la misma conexión TCP, y compresión gzip para reducir el tamaño de payloads con textos largos en contextos de RAG.
- **SOAP/XML:** consumido mediante zeep en Python o JAX-WS en Java, con Apache Camel como middleware de integración que absorbe la complejidad del protocolo y expone REST al sistema de IA; el WSDL como única fuente de verdad del contrato.
- **gRPC con Protocol Buffers:** .proto schemas versionados en un schema registry (Buf Registry o Confluent Schema Registry), generación automática de clientes en Python (grpcio), Go, Java, y otros lenguajes, con streaming bidireccional para respuestas de LLM en tiempo real.
- **Apache Kafka como integration backbone:** topics particionados por dominio de negocio para paralelismo, Consumer Groups para escalabilidad horizontal de los workers de IA, y schemas Avro en Confluent Schema Registry para compatibilidad backward entre versiones del productor y el consumidor.
- **File-based integration para mainframes:** SFTP con PGP encryption para transferencia segura de archivos fixed-width generados por COBOL, procesados por

pipelines de Spark (para volúmenes grandes) o Pandas (para volúmenes moderados) con detección de cambios por checksum para no reprocesar archivos ya procesados.

Idea central: La selección del protocolo de integración debe basarse en las capacidades del sistema legacy, no en las preferencias tecnológicas del equipo de IA. Integrar correctamente un sistema SOAP entrega valor de negocio inmediatamente; esperar a que el sistema legacy adopte REST puede no ocurrir nunca.

Con los protocolos de comunicación claros, la sección siguiente aborda la dimensión organizacional más delicada de la integración con legacy: cómo modernizar gradualmente los sistemas existentes sin interrumpir las operaciones de negocio que dependen de ellos.

Módulo 11 – Capítulo 03 – Sección 05

Gestión del cambio técnico: actualizar el legado gradualmente sin interrumpir el negocio

La modernización de sistemas legacy en organizaciones enterprise sigue una dinámica que cualquier ingeniero que haya trabajado en este contexto reconoce: hay consenso sobre que el sistema legacy necesita modernizarse, pero no hay acuerdo sobre cómo hacerlo sin interrumpir el negocio que depende de él, y la solución de "apagar el legacy y lanzar el nuevo sistema en una fecha acordada" invariablemente produce proyectos que se extienden años y terminan en rollbacks traumáticos. El big bang rewrite es el patrón de modernización que más frecuentemente fracasa en enterprise, y sin embargo es el que más frecuentemente se propone.

El patrón Strangler Fig — nombrado en referencia a la higuera estranguladora que crece alrededor de un árbol huésped hasta reemplazarlo — ofrece la alternativa incremental: en lugar de reemplazar el sistema legacy de golpe, se introduce una capa de abstracción (el facade) que inicialmente redirige el 100% del tráfico al sistema legacy, y paulatinamente se migran módulos funcionales al nuevo sistema mientras el facade gestiona el routing. El resultado es que en cualquier momento durante la migración el sistema está completamente funcional: el 20% del tráfico puede estar siendo servido por el nuevo sistema y el 80% por el legacy, y si aparecen problemas en el nuevo sistema, el facade revierte al legacy en minutos sin que los usuarios noten la transición.

La implementación técnica del Strangler Fig para integración con IA añade una dimensión específica: los feature flags. LaunchDarkly, Unleash, o un sistema propio basado en Redis permiten activar la nueva integración para un porcentaje configurable del tráfico, incrementando gradualmente ese porcentaje a medida que las métricas de calidad confirman que la nueva integración produce resultados equivalentes o mejores que el legacy. El rollout gradual — 5% del tráfico en la primera semana, 20% en la segunda si las métricas son estables, 50% en la tercera, y 100% en la cuarta — es el mecanismo que convierte una decisión binaria (legacy vs. nuevo) en un proceso continuo de validación con datos reales.

Las pruebas de regresión automatizadas son imprescindibles en este proceso. Un test suite que valida el comportamiento del sistema legacy antes y después de cada cambio de integración es la única forma de detectar regresiones antes de que lleguen a producción. Las **contract tests** implementadas con Pact o Spring Cloud Contract van más allá: verifican que el productor (el sistema legacy) y el consumidor (el sistema de IA) mantienen la compatibilidad de interfaces en cada despliegue, sin necesidad de ejecutar ambos sistemas en un entorno integrado para hacer la verificación.

La coordinación con el Change Advisory Board (CAB) es la dimensión organizacional de este proceso que los equipos técnicos frecuentemente subestiman. El CAB no es un obstáculo burocrático: es el mecanismo que protege al negocio de cambios no coordinados que pueden tener efectos en cascada sobre sistemas interdependientes. Preparar un RFC técnico bien documentado — con análisis de impacto sobre los sistemas dependientes, plan de rollback con tiempo de ejecución medido en staging, criterios de éxito cuantificables, y notificación a todos los equipos afectados — es la inversión que más frecuentemente acelera la aprobación del CAB.

Puntos críticos de la gestión del cambio

- **Strangler Fig con routing por feature flags:** facade que proxee el 100% al legacy inicialmente, con routing gradual al nuevo sistema mediante porcentajes controlados por feature flags (LaunchDarkly, Unleash), con capacidad de rollback inmediato modificando un valor de configuración.
- **Shadow mode deployment:** ejecutar el nuevo sistema de integración en paralelo al legacy, comparando las respuestas de ambos para detectar discrepancias sin impacto en el usuario final, antes de activar el corte de tráfico al nuevo sistema.
- **Contract testing con Pact:** pruebas de contrato que verifican la compatibilidad de interfaz entre el productor (legacy) y el consumidor (sistema de IA) en cada despliegue, detectando incompatibilidades antes de que lleguen a producción.
- **Feature flags para rollout incremental:** 5% → 20% → 50% → 100% del tráfico en intervalos de una semana, con criterios de progresión automática basados en métricas de calidad (latencia p95, tasa de error, calidad de respuesta del sistema de IA) y rollback automático si las métricas degradan.
- **Documentación de change requests para CAB:** RFC técnicos con análisis de impacto sobre sistemas downstream, plan de rollback probado y ejecutado en staging con tiempo de ejecución documentado (objetivo: menos de 15 minutos), y notificación a todos los equipos con sistemas dependientes con mínimo 5 días de antelación.

Buena práctica: Nunca realizar el corte de un sistema legacy a producción sin haber ejecutado previamente un rollback simulado completo en staging. Si el plan de rollback no puede ejecutarse en menos de 15 minutos en staging, no está listo para producción. La velocidad del rollback es tan importante como la calidad del nuevo sistema.

La sección de cierre del capítulo articula el argumento central: los sistemas legacy no son obstáculos a eliminar sino activos a integrar inteligentemente, y el AI Engineer que domina los patrones de integración con legado es sustancialmente más efectivo en contextos enterprise que el que los evita.

Módulo 11 – Capítulo 03 – Sección 06

Cierre: los sistemas legacy son una realidad — la integración inteligente es mejor que la sustitución apresurada

Los sistemas legacy no son obstáculos a eliminar sino activos de negocio que contienen décadas de lógica de dominio codificada, datos históricos irremplazables, e integraciones con proveedores y clientes construidas y estabilizadas durante años. La lógica de negocio que reside en 500.000 líneas de COBOL en el sistema de facturación bancaria no es código que puede replicarse correctamente en semanas: refleja reglas de negocio acumuladas durante 20 años de operación, muchas de ellas no documentadas en ningún lugar excepto en el comportamiento del sistema. Proponer el reemplazo completo como condición para implementar IA es, en la mayoría de los casos, bloquear indefinidamente el valor que la IA puede entregar.

Los patrones cubiertos en este capítulo — el Adapter SOAP-to-REST, la Facade de agregación, el Anti-Corruption Layer, el ETL y ELT con dbt, el CDC con Debezium, los protocolos REST/SOAP/gRPC/message queues según el contexto, y el Strangler Fig con feature flags — forman un toolkit completo para extraer el valor de los datos legacy e integrarlos con sistemas de IA modernos sin requerir la sustitución previa del sistema fuente. Cada patrón es independiente y componible: se puede usar Debezium para captura de cambios en una base de datos legacy mientras se usa SFTP para los archivos del mainframe y SOAP-to-REST para el sistema ERP, todo coordinado por el mismo pipeline de Kafka y normalizado por el mismo Anti-Corruption Layer.

La integración inteligente produce resultados incrementales que tienen valor por sí mismos: cada capa de abstracción construida sobre el legacy reduce el acoplamiento entre los sistemas de IA y los sistemas fuente, mejora la calidad de los datos disponibles mediante la normalización del ACL, y crea una base técnica que eventualmente puede facilitar la sustitución del legacy — pero de manera gradual, cuando el negocio está listo, y no como prerrequisito para empezar a entregar valor con IA. El AI Engineer enterprise que domina los patrones de integración con legado puede entregar los primeros resultados de IA en semanas,

mientras el equipo que insiste en la sustitución del legacy como prerequisite puede esperar años.

El siguiente capítulo eleva la perspectiva desde la integración con sistemas específicos hasta la gestión de múltiples tenants sobre la misma plataforma: cómo garantizar que los datos, el comportamiento, y los costos de cada cliente o unidad de negocio estén correctamente aislados y atribuidos en un sistema compartido.

"Cualquier organización que diseña un sistema produce inevitablemente un diseño cuya estructura es una copia de la estructura de comunicación de la organización." — Melvin Conway, informático, cuya ley aplica con igual fuerza a los sistemas legacy que a los nuevos: para integrarlos, primero hay que entender la organización que los creó.